

Universal Bridge Math Set

URIB Stack — Mathematical Foundations & Implementation Reference

Classification: Engineering Specification | Audience: Senior Engineers · Protocol Architects · Financial System Designers

Document Date: 09 July 2026 | Status: Technical Reference

This document defines the complete mathematical framework underlying the Universal Rail Integration Bridge (URIB) stack. It covers canonical tokenization, semantic meaning graphs, the ThreadZero truth chain, Bitcoin Taproot commitments with satoshi-level ordinal tracking, and multi-rail settlement invariants. Each section provides labeled constructs, formal definitions, and implementation-ready task breakdowns.

TABLE OF CONTENTS

§ 1 Canonical Tokenization — Structure + Permanence

1.1 Document Representation · 1.2 Deterministic Serialization · 1.3 Hashing for Permanence · 1.4 Versioning / Lineage

§ 2 Semantic Tokenization — Meaning Layer

2.1 Semantic Graph / Ontology · 2.2 Semantic Mapping · 2.3 Semantic Invariants · 2.4 Semantic Hash / Commitment

§ 3 ThreadZero — Truth + Lineage

3.1 Event Model · 3.2 Ordered Event Stream · 3.3 Thread Hash Chain · 3.4 Cross-Rail Projection

§ 4 Taproot + Satoshi Ordinal Tracking — Bitcoin Layer

[4.1 Taproot Tweak](#) · [4.2 Ordinal Assignment](#) · [4.3 Satoshi-Level Binding](#) · [4.4 Commit-Reveal Pattern](#)

§ 5 Universal Bridge — URIB Stack

[5.1 Rail Set](#) · [5.2 Rail State Mapping](#) · [5.3 Cross-Rail Invariants](#) · [5.4 Identity Continuity](#)

§ 6 Distilled Implementation Checklist

§ 7 Architecture Diagram — Literal Stack View

§ 8 Full Implementation Plan

[8.1 Overall Goal](#) · [8.2 Core Data Structures](#) · [8.3 Core API Surface](#) · [8.4 Implementation Phases](#) · [8.5](#)

[Orchestration Pipeline](#)

§ 9 Module Reference

[canonical](#) · [semantic](#) · [threadzero](#) · [stack](#) · [bitcoin](#) · [urib](#) · [settlement](#)

§ 1 Canonical Tokenization — Structure + Permanence

The document and state backbone. Provides math for deterministic identity and permanence of real-world artifacts. Canonical tokenization ensures that every real-world document — contract, lease, deed, or settlement — can be represented, serialized, hashed, and chained in a form that is fully reversible, order-stable, and cryptographically anchored across all downstream rails.

1.1 Document Representation

LABEL: Canonical Document Model

A document **D** becomes a structured object composed of typed, constrained, semantically meaningful fields:

DEFINITION — Document Structure

$D = \{ f_1, f_2, \dots, f_n \}$ where each f_i is a field with:

- type — data type (string, decimal, date, reference, ...)
- constraints — validation rules and bounds
- semantics — ontological meaning within the domain

1.2 Deterministic Serialization

LABEL: Canonical Serialization

A deterministic serialization function maps a document to a byte string that can be unambiguously reconstructed and compared across all systems:

DEFINITION — Serialization Function

$\text{canon_serialize}(D) \rightarrow B$ where B is a byte string satisfying:

- order-stable — field ordering is fully deterministic under the schema
- schema-bound — structure enforced by the canonical schema definition
- reversible — B uniquely determines D with no information loss

1.3 Hashing for Permanence

LABEL: Canonical Hash

A cryptographic hash of the serialized document byte string serves as the document's permanent, system-wide fingerprint:

DEFINITION — Canonical Hash

$h_D = H(B)$ where H is SHA-256, and h_D is used as:

- Document fingerprint for immutability verification
- Anchor into Bitcoin / XRP / ISO rails
- Identity key for reconstruction and deduplication

1.4 Versioning / Lineage

LABEL: Canonical Lineage

Document versions are chained cryptographically, ensuring a tamper-evident, append-only history of all mutations:

DEFINITION — Lineage Chain

$h_D_0 \rightarrow h_D_1 \rightarrow \dots \rightarrow h_D_k$ with each successor defined as: $h_D(i+1) = H(h_D_i \parallel \Delta_i)$ where:

- h_D_i — hash of document at version i
- Δ_i — change set (diff) applied at step i
- $\parallel$ — byte-level concatenation operator

► SECTION SUMMARY

"Provides math for permanence + reversibility + identity of real-world artifacts."

§ 2 Semantic Tokenization — Meaning Layer

The "what does this actually mean?" layer. Semantic tokenization lifts raw document structure into a relational graph encoding parties, obligations, assets, and rights — preserving logical invariants that must hold across all rail representations of the same document.

2.1 Semantic Graph / Ontology

LABEL: Semantic Graph

The document's meaning is encoded as a directed, typed graph over domain entities:

DEFINITION — Semantic Graph

$G = (V, E)$ where: V = nodes — parties, contracts, assets, obligations E
= edges — typed relations between nodes:
· owns — party holds asset
· owes — obligation from A to B
· guarantees — party backs obligation
· settles — event discharges obligation
· references — document references another document

2.2 Semantic Mapping

LABEL: Semantic Mapping

A functional mapping transforms a canonical document into its induced semantic graph:

DEFINITION — Semantic Mapping Function

$\text{sem_map}(D) \rightarrow G_D$ where G_D is the semantic graph induced by document D :
· All parties, roles, and entities are lifted to nodes V
· All obligations,

rights, and references become typed edges E · The mapping is deterministic and schema-driven

2.3 Semantic Invariants

LABEL: Invariant Constraints

The semantic graph must satisfy logical and relational invariants that enforce correctness across all rail representations:

DEFINITION — Semantic Invariants

$\sum \text{debts} = \sum \text{credits}$ [Value Conservation] $\text{obligation}(A, B) \Rightarrow \text{right}(B, A)$ [Duality Invariant] These invariants must hold across: ·
All rail projections of G_D · All versioned states in the lineage chain · All cross-rail settlement states

2.4 Semantic Hash / Commitment

LABEL: Semantic Commitment

The semantic graph is serialized and hashed to prove that meaning — not merely bytes — is preserved across all transformations:

DEFINITION — Semantic Hash

$h_G = H(\text{canon_serialize}(G_D))$ Semantics: · Proves the meaning encoded in G_D is preserved, not just raw bytes · h_G is anchored alongside

h_D in the stack commitment · Changes to meaning produce a distinct h_G even with identical h_D

► SECTION SUMMARY

"Provides math for meaning-preserving identity resolution."

§ 3 ThreadZero — Truth + Lineage

The deterministic truth spine of the URIB stack. ThreadZero captures every event in a causally ordered, cryptographically chained sequence, producing a single truth anchor T^* that can be independently verified and projected onto any downstream rail.

3.1 Event Model

LABEL: Canonical Event Model

Each atomic event in the system is represented as a structured, timestamped record:

DEFINITION — Event Structure

$E_i = \{ t_i, \text{actor}_i, \text{action}_i, \text{payload}_i \}$ where: · t_i — deterministic timestamp (logical or wall-clock) · actor_i — DID of the initiating party · action_i — typed action descriptor · payload_i — canonical serialization of event data

3.2 Ordered Event Stream

LABEL: Thread Sequence

The complete, causally ordered sequence of all events forms the ThreadZero truth record:

DEFINITION — Thread Sequence

$T = (E_0, E_1, \dots, E_n)$ Properties:

- Causally ordered — no event appears before its preconditions
- Append-only — events are never removed or reordered
- Deterministic — given T , all rail states are fully reconstructable

3.3 Thread Hash Chain

LABEL: ThreadZero Hash Chain

Events are chained cryptographically to produce a single, irrefutable truth anchor:

DEFINITION — Thread Hash Chain

$T_0 = H(E_0)$ $T(i+1) = H(T_i \parallel E(i+1))$ $T^* = T_n \leftarrow$ truth anchor for the entire history

Properties:

- Any modification of any event E_i invalidates all $T(j)$ for $j \geq i$
- T^* is the canonical integrity proof for the complete event history
- T^* is embedded in the stack commitment C_{stack}

3.4 Cross-Rail Projection

LABEL: Rail Projection

Each rail receives a deterministic projection of the ThreadZero sequence — a filtered, rail-relevant subset that remains anchored to the global truth:

DEFINITION — Rail Projection Function

$\pi_r : T \rightarrow T_r$ where:

- T_r is the subsequence of events relevant to rail r
- $r \in \{ \text{Bitcoin, XRP, ISO 20022, CBDC, ...} \}$
- All T_r are verifiably derived from and anchored to T^*
- Projection is deterministic and reproducible

► SECTION SUMMARY

"Provides math for deterministic, reconstructable truth across all rails."

§ 4 Taproot + Satoshi Ordinal Tracking — Bitcoin Layer

Bitcoin commitment and satoshi-level identity layer. This section defines the elliptic curve math for Taproot key tweaking, the ordinal indexing scheme for satoshi-level tracking, and the commit-reveal pattern that anchors the full URIB stack state into Bitcoin's UTXO model.

4.1 Taproot Tweak

LABEL: Taproot Commitment

The Taproot internal key is tweaked with a commitment derived from the canonical, semantic, and thread state of the stack:

DEFINITION — Taproot Key Tweak

$P' = P_{\text{int}} + H(P_{\text{int}} \parallel C) \cdot G$ where: G — elliptic curve generator point (secp256k1) P_{int} — internal Taproot public key C — stack commitment (see Section 4 below): $C = H(h_D \parallel h_G \parallel T^*)$ P' — tweaked output public key committed to Bitcoin chain Note: H in the Taproot context is tagged SHA-256 per BIP-341.

4.2 Ordinal Assignment

LABEL: Ordinal Index

Each satoshi in a UTXO is assigned a globally unique ordinal index using FIFO ordering rules, enabling satoshi-level identity and tracking:

DEFINITION — Ordinal Assignment

$\text{ord_map}(\text{UTXO}) \rightarrow \{ s_j : j \in [\text{offset}, \text{offset} + \text{value}) \}$ Rules: · FIFO assignment — inputs are consumed in order; outputs assigned in order · Each satoshi s_j carries a globally unique index j from genesis · $\text{UTXO} \rightarrow \text{ordinal mappings}$ are deterministically reconstructable from the full Bitcoin chain history

4.3 Satoshi-Level Binding

LABEL: Satoshi Binding

Each tracked satoshi is bound to the full canonical, semantic, and truth state — enabling satoshi-level tokenization of real-world artifacts:

DEFINITION — Satoshi Binding

$\text{bind}(s_j) = (s_j, h_D, h_G, T^*)$ where: · s_j — globally unique satoshi ordinal index · h_D — canonical document hash (Section 1.3) · h_G — semantic graph hash (Section 2.4) · T^* — ThreadZero truth anchor (Section 3.3) This tuple is the satoshi-level tokenization of the real-world artifact.

4.4 Commit-Reveal Pattern

LABEL: Commit / Reveal Math

The two-phase commit-reveal pattern anchors bridge and tokenization state into Bitcoin transactions:

DEFINITION — Commit / Reveal Transactions

tx_commit: · Encodes the bridge / tokenization state $C = H(h_D \parallel h_G \parallel T^*)$
· Commits to $P' = P_int + H(P_int \parallel C) \cdot G$ as the output key · Broadcast and
mined without revealing internal structure tx_reveal: · Reveals the Taproot
script path encoding the bridge state · Satisfies the Taproot spending
condition using the script · The script encodes your full bridge state
commitment · Provides on-chain verifiability of the full URIB stack state

► SECTION SUMMARY

"Provides math for Bitcoin-anchored, satoshi-level identity and commitments."

§ 5 Universal Bridge — URIB Stack

Where all layers are stitched together. The URIB stack takes canonical, semantic, and truth state as inputs and produces rail-specific state representations that are value-equivalent, identity-continuous, and cryptographically provable across all settlement rails.

5.1 Rail Set

LABEL: Rail Enumeration

DEFINITION — Rail Set

$R = \{ \text{BTC, XRP, ISO, CBDC, ...} \}$ Each rail $r \in R$ has:

- A distinct settlement mechanism and finality model
- A rail-specific identity namespace
- A defined state schema compatible with Φ_r

5.2 Rail State Mapping

LABEL: Rail State

For each rail r , the rail state is derived deterministically from the canonical document, its semantic graph, and the ThreadZero event sequence:

DEFINITION — Rail State Mapping

$S_r = \Phi_r(D, G_D, T)$ where the mapping Φ_r respects:

- Canonical structure — same document produces same rail state
- Semantic meaning — obligations and rights preserved in S_r
- Thread truth — all state changes traceable via T^*

5.3 Cross-Rail Invariants

LABEL: Bridge Invariants

Settlement equivalence is enforced via cross-rail proofs that guarantee value and semantic fidelity across all rail representations:

DEFINITION — Cross-Rail Invariants

Value equivalence: $\text{val}(S_BTC) = \text{val}(S_XRP) = \text{val}(S_ISO)$ Semantic
equivalence: $\text{sem}(S_BTC) \equiv \text{sem}(S_XRP) \equiv \text{sem}(S_ISO)$ [up to rail encoding]
Settlement equivalence: Enforced via cross-rail proofs anchored to T^* and
 C_stack

5.4 Identity Continuity

LABEL: Identity Mapping

The same real-world party maintains a continuous, verifiable identity across all rails through the DID-to-rail mapping function:

DEFINITION — Identity Mapping

$\Psi_r(\text{DID}) \rightarrow \text{rail-specific identity}$ where: · DID is the Decentralized Identifier of the party · Ψ_r maps DID to the rail-native identity representation · The mapping is injective — the same party is the same entity across all rails, with no identity fragmentation

► SECTION SUMMARY

"The math that makes URIB a rail-agnostic, deterministic, identity-aware settlement fabric."

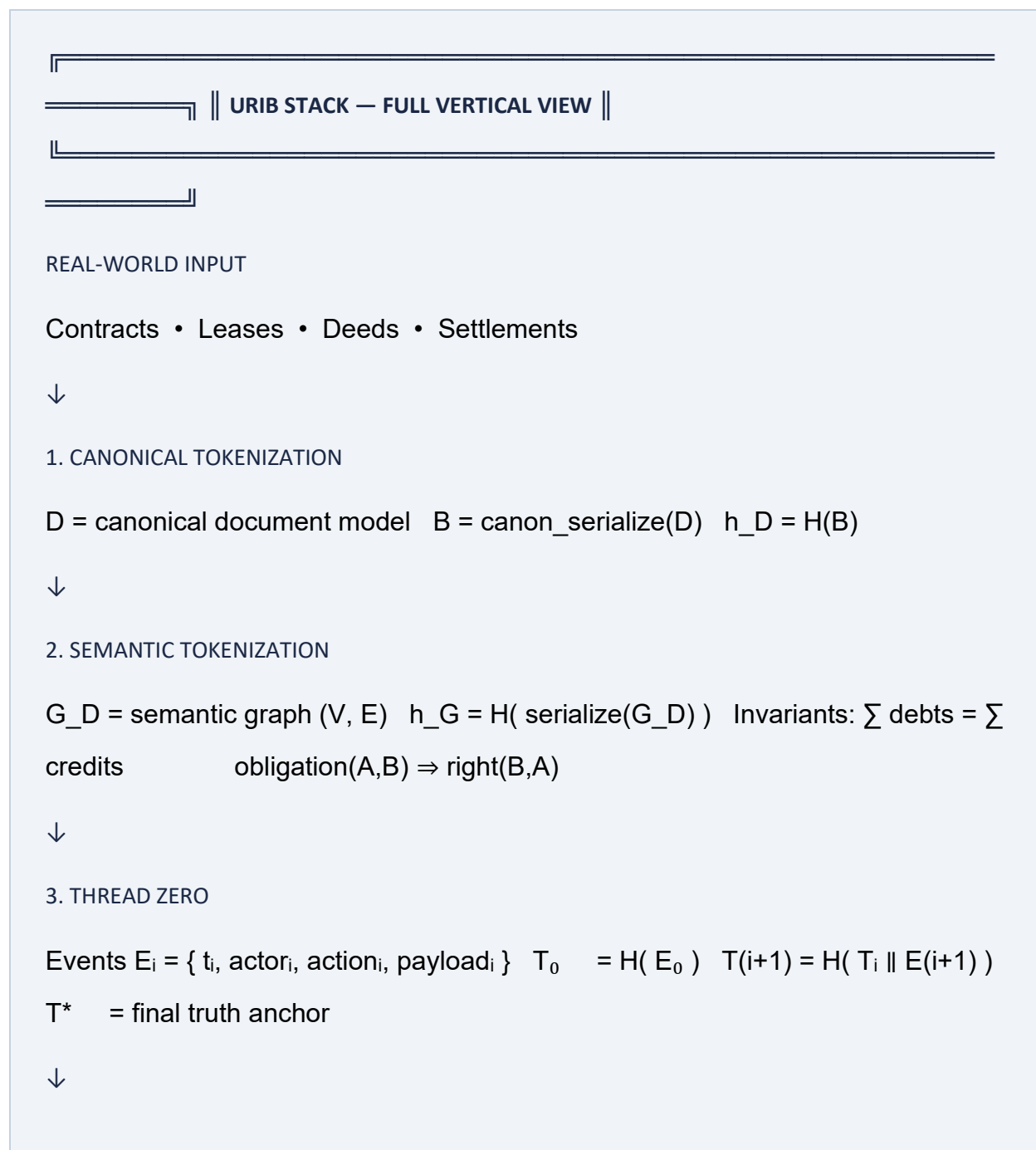
§ 6 Distilled Implementation Checklist

The following table summarizes all mathematical constructs and engineering components required for a complete URIB stack implementation. Each row maps directly to a subsection of this specification and one or more implementation tasks in Section 8.

Component	Description
h_D, h_G, T^*, C_{stack}	Core cryptographic hashes for documents (h_D), semantic graphs (h_G), thread truth anchor (T^*), and the unified stack commitment (C_{stack})
Serialization	Canonical, reversible, schema-bound serialization functions — the foundation of determinism across all layers
Graphs / Relations	Semantic graph G_D with typed node and edge sets (V, E) and enforced invariants over all rail projections
Hash Chains	ThreadZero hash chain from genesis anchor T_0 through to the final truth anchor T^*
Elliptic Curve Math	Taproot key tweak: $P' = P_{int} + H(P_{int} \parallel C) \cdot G$ — BIP-341 compliant secp256k1 commitment
Ordinal Indexing	FIFO satoshi mapping via $ord_map(UTXO)$ and satoshi binding $bind(s_j)$ to canonical and semantic state
Cross-Rail Mappings	State mapping function Φ_r and identity mapping function Ψ_r for each rail $r \in R$
Invariants	Value conservation, semantic equivalence, and settlement equivalence enforced across all rails via cross-rail proofs

§ 7 Architecture Diagram — Literal Stack View

The following diagram presents the complete URIB stack as a vertical processing pipeline — from raw real-world input at the top through to deterministic, multi-rail settlement output at the bottom. Each layer corresponds to one numbered section of this specification.



4. STACK COMMITMENT

$$C_stack = H(h_D \parallel h_G \parallel T^*)$$

↓

5. BITCOIN: TAPROOT + ORDINALS

$$P' = P_int + H(P_int \parallel C_stack) \cdot G \quad \text{ord_map(UTXO)} \rightarrow \text{satoshis } s_j \quad \text{bind}(s_j) = (s_j, h_D, h_G, T^*)$$

↓

6. URIB — THE BRIDGE

$$\text{Rails } R = \{ \text{BTC, XRP, ISO, CBDC, ...} \} \quad S_r = \Phi_r(D, G_D, T) \quad \text{for each } r \in R$$

Identity continuity via $\Psi_r(\text{DID})$ $C_bridge = H(S_BTC \parallel S_XRP \parallel S_ISO \dots)$

↓

7. OUTPUT / SETTLEMENT

BTC → Taproot output (P') XRP → settlement state (S_r) ISO → 20022 pacs.008 messages TZ → ThreadZero audit trail

to: T* and C_stack

§ 8 Full Implementation Plan

8.1 Overall Goal

Implement a modular, layered system that performs all of the following operations in a deterministic, auditable, and interoperable manner:

- Canonicalize real-world documents into schema-bound, reversible canonical objects
- Build semantic meaning graphs with enforced invariants over all rail representations
- Maintain a ThreadZero truth chain as an append-only, cryptographically linked event log
- Commit stack state into Bitcoin via Taproot key tweaking and ordinal satoshi binding
- Map stack state across multiple rails (BTC, XRP, ISO, CBDC) via the URIB layer
- Emit deterministic, independently verifiable settlement outputs per rail with a complete ThreadZero audit trail

8.2 Core Data Structures

Type	Structure Definition
Document	<code>{ fields: List<Field>, schema_id: String }</code>
Field	<code>{ name: String, type: String, value: Any }</code>
SemanticGraph	<code>{ nodes: List<Node>, edges: List<Edge> }</code>
Node	<code>{ id: String, type: String, attrs: Map<String, Any> }</code>
Edge	<code>{ from: String, to: String, relation: String }</code>
Event	<code>{ timestamp: Time, actor: String, action: String, payload: Any }</code>
ThreadState	<code>{ events: List<Event>, anchor: Hash }</code>
RailState	<code>{ rail_id: String, state: Any, value: Decimal }</code>
DID	<code>{ id: String }</code>
RailIdentity	<code>{ rail_id: String, did: DID, rail_specific_id: String }</code>

8.3 Core API Surface

Canonical Layer

MODULE: canonical

canonicalize_document(raw_doc)	→ Document	serialize_document(doc)
→ Bytes	hash_document(bytes)	→ Hash

Semantic Layer

MODULE: semantic

build_semantic_graph(doc)	→ SemanticGraph
check_semantic_invariants(graph)	→ Bool
serialize_semantic_graph(graph)	→ Bytes
hash_semantic_graph(bytes)	→ Hash

ThreadZero Layer

MODULE: threadzero

create_event(actor, action, payload)	→ Event	append_event(thread, event)
→ ThreadState	compute_thread_anchor(thread)	→ Hash

Stack Commitment

MODULE: stack

compute_stack_commitment(h_doc, h_sem, t_anchor) → Hash

Bitcoin Layer

MODULE: bitcoin

compute_taproot_key(P_int, C_stack) → PubKey
assign_ordinals(utxo) → List

<

SatoshiOrdinal

>

bind_satoshis(ordinals, h_doc, h_sem, t_anchor) → List

<

Binding

>

URIB Layer

MODULE: urib

map_to_rail_state(rail_id, doc, graph, thread) → RailState

map_identity_to_rail(did, rail_id) → RailIdentity

enforce_cross_rail_invariants(states) → Bool

compute_bridge_commitment(states) → Hash

Settlement Layer

MODULE: settlement

emit_btc_taproot_tx(P_prime, bindings) → BtcTx emit_xrp_settlement(state)

→ XrpTx emit_iso20022_messages(state) → List

<

IsoMessage

>

emit_threadzero_audit(thread) → AuditLog

8.4 Implementation Phases

Phase 1 — Canonical + Semantic Core

- Task 1.1: Implement `Document`, `Field`, serialization (`serialize_document`), and hashing (`hash_document`)
- Task 1.2: Implement `SemanticGraph`, node/edge extraction via `build_semantic_graph`
- Task 1.3: Implement `check_semantic_invariants`, `serialize_semantic_graph`, and commitment hashing via `hash_semantic_graph`

Phase 2 — ThreadZero Truth Engine

- Task 2.1: Implement `Event` and `ThreadState` data structures
- Task 2.2: Implement hash-chain logic for ThreadZero via `append_event` and `compute_thread_anchor`
- Task 2.3: Integrate events with document and semantic changes — emit events on canonical and semantic mutations

Phase 3 — Stack Commitment + Bitcoin Layer

- Task 3.1: Implement `compute_stack_commitment(h_doc, h_sem, t_anchor)` — SHA-256 over concatenated hashes
- Task 3.2: Implement Taproot tweak math and key derivation via `compute_taproot_key` per BIP-341
- Task 3.3: Implement ordinal tracking via `assign_ordinals` and satoshi binding via `bind_satoshis`

Phase 4 — URIB Rail Mapping

- Task 4.1: Implement `RailState` and `map_to_rail_state` for BTC, XRP, and ISO 20022 rails
- Task 4.2: Implement DID → rail identity mapping via `map_identity_to_rail`
- Task 4.3: Implement cross-rail invariants via `enforce_cross_rail_invariants` and bridge commitment via `compute_bridge_commitment`

Phase 5 — Settlement + Audit

- Task 5.1: Implement BTC, XRP, and ISO output adapters: `emit_btc_taproot_tx`, `emit_xrp_settlement`, `emit_iso20022_messages`
- Task 5.2: Implement ThreadZero audit log export via `emit_threadzero_audit`
- Task 5.3: Integrate all layers into a single orchestrated pipeline with end-to-end determinism

8.5 End-to-End Orchestration Pipeline

The following pseudocode defines the complete orchestration pipeline — from raw real-world document input to final settlement output emission across all rails:

```
— URIB ORCHESTRATION PIPELINE —————

function
orchestrate(raw_doc, actor, did):

— STAGE 1: Canonical Layer —————

doc    = canonicalize_document(raw_doc)  bytes_d = serialize_document(doc)
h_doc  = hash_document(bytes_d)

— STAGE 2: Semantic Layer —————

graph  = build_semantic_graph(doc)  assert check_semantic_invariants(graph)
bytes_g = serialize_semantic_graph(graph)  h_sem  =
hash_semantic_graph(bytes_g)

— STAGE 3: ThreadZero Layer —————

evt    = create_event(actor, "TOKENIZE", { h_doc, h_sem })  thread =
append_event(thread, evt)  t_anchor = compute_thread_anchor(thread)

— STAGE 4: Stack Commitment —————

C_stack = compute_stack_commitment(h_doc, h_sem, t_anchor)

— STAGE 5: Bitcoin Layer —————

P_prime = compute_taproot_key(P_int, C_stack)  ordinals =
assign_ordinals(utxo)  bindings = bind_satoshis(ordinals, h_doc, h_sem,
t_anchor)

— STAGE 6: URIB Rail Mapping —————
```

```
states = {} for r in R:
```

```
// R = { BTC, XRP, ISO, CBDC, ... }
```

```
states[r] = map_to_rail_state(r, doc, graph, thread) identity_r =
```

```
map_identity_to_rail(did, r) assert enforce_cross_rail_invariants(states)
```

```
C_bridge = compute_bridge_commitment(states)
```

```
— STAGE 7: Settlement Emission —————
```

```
btc_tx = emit_btc_taproot_tx(P_prime, bindings) xrp_tx =
```

```
emit_xrp_settlement(states[XRP]) iso_msgs =
```

```
emit_iso20022_messages(states[ISO]) audit = emit_threadzero_audit(thread)
```

```
return { btc_tx, xrp_tx, iso_msgs, audit, C_stack, C_bridge }
```

```
— END PIPELINE —————
```

§ 9 Module Reference

The following module cards provide a complete reference for each layer of the URIB stack. Each card includes the module's purpose, its owned types, and its full function signatures as defined in Section 8.3.

canonical

Turn real-world artifacts into deterministic, reversible, hashable canonical objects.

TYPES

- `Document { fields: List<Field>, schema_id: String }`
- `Field { name: String, type: String, value: Any }`

FUNCTIONS

- `canonicalize_document(raw_doc: Any) → Document`
- `serialize_document(doc: Document) → Bytes`
- `hash_document(bytes: Bytes) → Hash`

semantic

Extract and preserve meaning as a typed graph with enforced logical and relational invariants.

TYPES

- `SemanticGraph { nodes: List<Node>, edges: List<Edge> }`
- `Node { id: String, type: String, attrs: Map<String, Any> }`
- `Edge { from: String, to: String, relation: String }`

FUNCTIONS

- `build_semantic_graph(doc: Document) → SemanticGraph`
- `check_semantic_invariants(graph: SemanticGraph) → Bool`
- `serialize_semantic_graph(graph: SemanticGraph) → Bytes`
- `hash_semantic_graph(bytes: Bytes) → Hash`

threadzero

Maintain deterministic, causally ordered, cryptographically chained truth over time.

TYPES

- `Event { timestamp: Time, actor: String, action: String, payload: Any }`
- `ThreadState { events: List<Event>, anchor: Hash }`

FUNCTIONS

- `create_event(actor: String, action: String, payload: Any) → Event`
- `append_event(thread: ThreadState, event: Event) → ThreadState`
- `compute_thread_anchor(thread: ThreadState) → Hash`

stack

Bind canonical, semantic, and truth state into a single unified cryptographic commitment.

TYPES

- Hash – 32-byte SHA-256 digest (represented as hex string)

FUNCTIONS

- `compute_stack_commitment(h_doc: Hash, h_sem: Hash, t_anchor: Hash) → Hash`

bitcoin

Anchor stack commitment into Bitcoin via Taproot key tweaking and satoshi-level ordinal binding.

TYPES

- PubKey – secp256k1 public key (BIP-340 x-only format)
- SatoshiOrdinal { index: UInt64, utxo_ref: String }
- Binding { ordinal: SatoshiOrdinal, h_doc: Hash, h_sem: Hash, t_anchor: Hash }

FUNCTIONS

- `compute_taproot_key(P_int: PubKey, C_stack: Hash) → PubKey`
- `assign_ordinals(utxo: UTXO) → List<SatoshiOrdinal>`
- `bind_satoshis(ordinals: List<SatoshiOrdinal>, h_doc: Hash,
h_sem: Hash, t_anchor: Hash) → List<Binding>`

urib

Map stack state across all settlement rails and enforce cross-rail value and identity invariants.

TYPES

- `RailState { rail_id: String, state: Any, value: Decimal }`
- `DID { id: String }`
- `RailIdentity { rail_id: String, did: DID, rail_specific_id: String }`

FUNCTIONS

- `map_to_rail_state(rail_id: String, doc: Document, graph: SemanticGraph, thread: ThreadState) → RailState`
- `map_identity_to_rail(did: DID, rail_id: String) → RailIdentity`
- `enforce_cross_rail_invariants(states: Map<String, RailState>) → Bool`
- `compute_bridge_commitment(states: Map<String, RailState>) → Hash`

settlement

Emit concrete, verifiable settlement outputs per rail and produce a complete ThreadZero audit trail.

TYPES

- `BtcTx` – signed Bitcoin transaction (Taproot, BIP-341)
- `XrpTx` – XRP Ledger transaction object
- `IsoMessage` – ISO 20022 pacs.008 message payload
- `AuditLog` – serialized ThreadZero event sequence with anchors

FUNCTIONS

- `emit_btc_taproot_tx(P_prime: PubKey, bindings: List<Binding>) → BtcTx`
- `emit_xrp_settlement(state: RailState) → XrpTx`
- `emit_iso20022_messages(state: RailState) → List<IsoMessage>`
- `emit_threadzero_audit(thread: ThreadState) → AuditLog`